

EasyService Marketplace Platform

Master Project & Testing Specification (v1.0)

Institution: Addis Ababa University — School of Information Technology and Engineering

Course: Software Testing and Validation

Instructor: Abel Tadesse

Deadline: Sunday, 13 September 2026, end of day

Document: Master Project & Testing Specification v1.0

Repository: <https://github.com/Abyman2/Easyservice>

Group Members

#	Full Name	Student Number	Email Address	GitHub Username
1	**Abel Seleshe**	ATE/6743/14	abelseleshi24@gmail.com	[Abyman2](https://github.com/Abyman2)
2	**Baheran Tesfaye**	ATE/5750/14	Bahrantesfaye1@gmail.com	[Bahrann](https://github.com/Bahrann)
3	**Nebiyu Yohannes**	ATE/3973/14	natijhonny@gmail.com	[NebiyuYohannes](https://github.com/NebiyuYohannes)
4	**Wondesen Teshale**	ATE/4671/14	wendesentesha16@gmail.com	[WondesenTeshale](https://github.com/WondesenTeshale)

1. Project Purpose & Philosophy

EasyService is an Ethiopia-first full-stack marketplace connecting customers with verified service providers across four primary categories:

1. **Hotels & Accommodation**
2. **Car Rentals**
3. **Event Tickets**
4. **Store Products & Services**

Development & Testing Philosophy

> Requirement → Risk Analysis → Formal Test Design → Test Doubles & Implementation → Automation → Continuous Integration → Defect Catch & Regression → Verification & Validation ■

EasyService is deliberately architected around testing requirements: lightweight in-memory persistence and fake external service doubles isolate business logic for deterministic, high-speed execution.

2. Target Users & Journeys

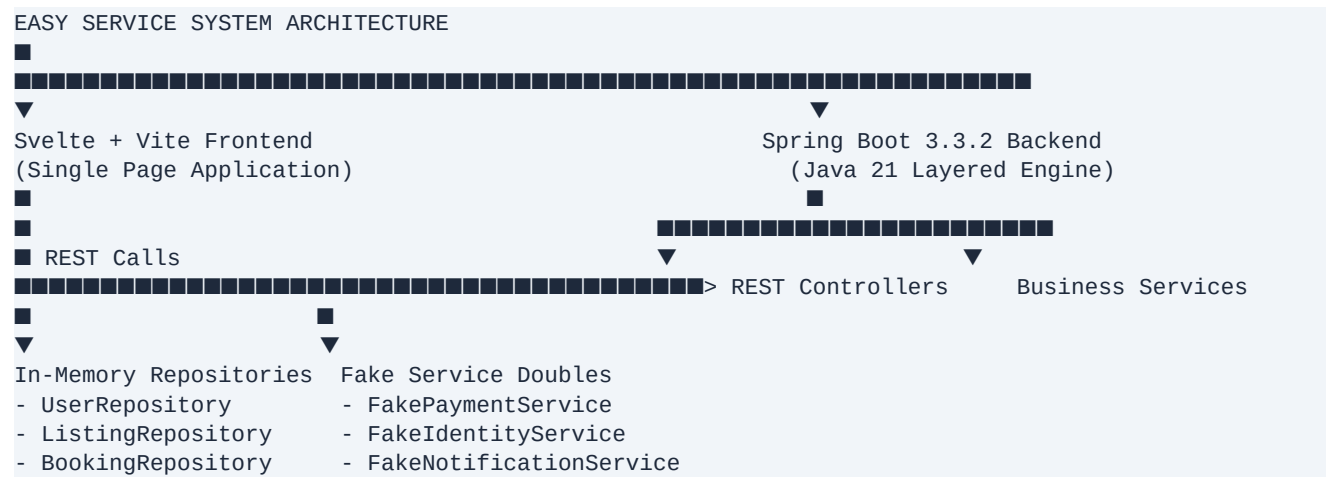
2.1 Customer Role

- **Ethiopian Customer:** Requires Name, Email, Ethiopian Phone, Fayda National ID (test value), Simulated Payment.
- **Foreign Customer:** Requires Name, Email, Country, Phone, Passport Number (test value), Simulated Payment.
- **Capabilities:** Register, Verify Identity, Login (Email/Phone), Browse & Filter Listings, Apply Promotions, Book/Purchase, View History, Cancel Eligible Transactions.

2.2 Provider Role

- **Capabilities:** Register/Login, Create Provider Profile, Manage Listings, Set Price & Capacity, Define Promotions/Hot Deals, Publish/Unpublish Listings, Monitor Incoming Transactions, Manage Transaction Status.

3. Technology Stack & Architectural Topology

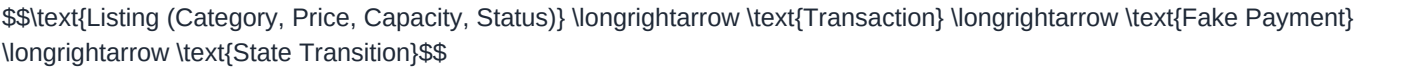


Stack Alignment

- **Backend:** Java 21, Spring Boot 3.3.2 (Web, Validation, Security).
- **Frontend:** Svelte + Vite SPA.
- **Unit Testing:** JUnit 5 + Mockito.
- **Integration Testing:** Spring Boot Test + MockMvc.
- **System / E2E Testing:** Selenium WebDriver with **Page Object Model (POM)**.
- **Code Coverage:** JaCoCo Maven Plugin (Target: 80% Branch Coverage).
- **Continuous Integration:** GitHub Actions + Jenkins (via Docker).

4. Shared Marketplace Domain Engine

All four categories (Hotel, Car, Event, Store) utilize the unified core marketplace engine:



Core Domain Entities

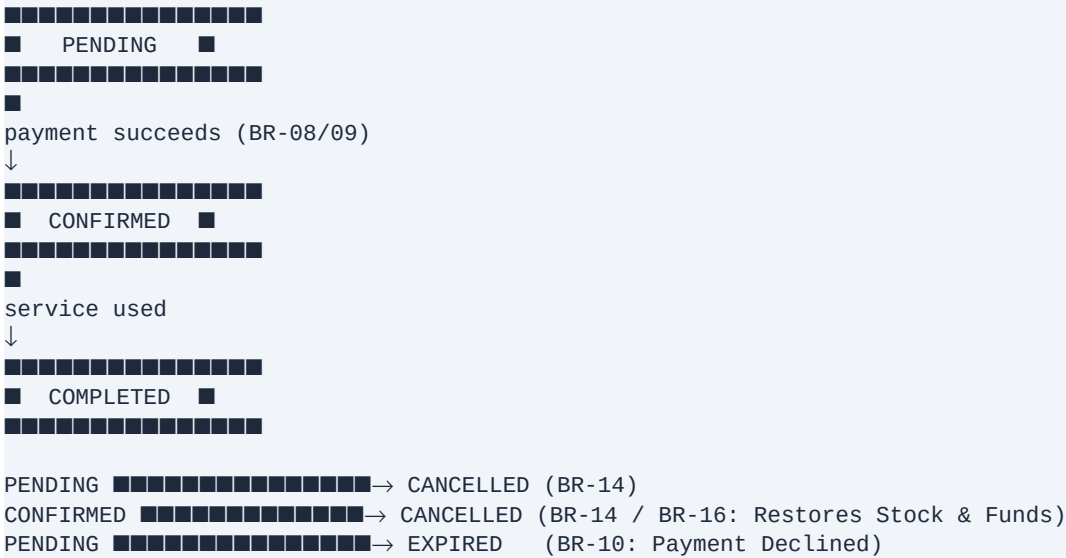
- User:** id, name, email, phone, password, country, customerType, identityType, identityStatus, balance.
- Provider:** id, businessName, name, email, phone, verified.
- Listing:** id, providerId, title, category, description, price, capacity, availableQuantity, status.
- Promotion:** id, listingId, discountPercentage, minimumAmount, startDate, endDate, status.
- Transaction:** id, customerId, listingId, quantity, totalAmount, status, createdAt, updatedAt.

5. Core Business Rules (BR-01 to BR-20)

Rule ID	Name	Formal Rule Specification
:---	:---	:---
BR-01	Identity Type	Ethiopian customer $\rightarrow$ Fayda; Foreign customer $\rightarrow$ Passport.
BR-02	Identity Verification	Only customers with `VERIFIED` identity status can initiate transactions.
BR-03	Authentication	Customers can authenticate via Email + Password OR Phone + Password.
BR-04	Listing Visibility	Only listings with `PUBLISHED` status are visible and bookable.
BR-05	Listing Price	Listing price must be strictly greater than zero ($> 0\$$).
BR-06	Booking Quantity	Transaction quantity must be strictly greater than zero ($> 0\$$).
BR-07	Inventory Capacity	Booking quantity cannot exceed `availableQuantity`.
BR-08	Payment Sufficiency	Customer must possess sufficient balance for transaction execution.
BR-09	Exact Balance	Payment succeeds when $\text{\text{Balance}} = \text{\text{Transaction Amount}}\$$.
BR-10	Insufficient Funds	Payment is declined and transaction marked `EXPIRED` when $\text{\text{Balance}} < \text{\text{Amount}}\$$.

BR-11	Promo Minimum	Promotion requires transaction amount to meet or exceed `minimumAmount`.
BR-12	Promo Validity	Expired or inactive promotions cannot be applied.
BR-13	Valid Transitions	Transactions move only through permitted state transitions.
BR-14	Cancellation State	Cancellation is allowed only from `PENDING` or `CONFIRMED` states.
BR-15	Stock Reservation	Successful booking reduces listing `availableQuantity` by quantity booked.
BR-16	Stock Restoration	Cancellation of `CONFIRMED` transaction restores `availableQuantity` and refunds balance.
BR-17	Listing Validity	Provider cannot publish incomplete or invalid listings.
BR-18	Provider Boundaries	Provider cannot modify or publish another provider's listing.
BR-19	Account Uniqueness	Duplicate registrations with existing Email or Phone are rejected.
BR-20	Role Security	Role-based authorization isolates Customer and Provider operational endpoints.

6. Transaction State Machine



Explicit Negative State Transition Assertions

- `\text{COMPLETED}` $\rightarrow$ `\text{Cancel}` `\text{CANCELLED}` $\rightarrow$ **FORBIDDEN (IllegalStateException)**
 - `\text{COMPLETED}` $\rightarrow$ `\text{Complete}` `\text{COMPLETED}` $\rightarrow$ **FORBIDDEN (IllegalStateException)**
 - `\text{EXPIRED}` $\rightarrow$ `\text{Confirm}` `\text{CONFIRMED}` $\rightarrow$ **FORBIDDEN (IllegalStateException)**
 - `\text{CANCELLED}` $\rightarrow$ `\text{Confirm}` `\text{CONFIRMED}` $\rightarrow$ **FORBIDDEN (IllegalStateException)**
-

7. Testing Strategy & Pyramid Breakdown

```

/ \
/   \      System / E2E Tier (Selenium + POM)
/ S   \      8-12 High-Value Browser Journeys
/-----\
/   I   \      Integration Tier (Spring Boot + MockMvc)
/-----\      15-25 Controller / Service / Repo Scenarios
/       U   \      Unit Tier (JUnit 5 + Mockito + Fakes)
/-----\      35+ Fast Business Rule Isolation Tests

```

7.1 Test Doubles Strategy

- **Fake:** `FakePaymentService` & `FakeIdentityVerificationService` provide full in-memory working logic with outcome toggles.
- **Mock/Spy:** `FakeNotificationService` verifies email/SMS dispatch invocations.
- **Controlled Randomness:** `FakeRandomNumberGenerator` eliminates non-deterministic randomness in the Random Payer Wheel tool.

7.2 Code Coverage & CI/CD Target

- **JaCoCo Minimum Threshold:** $\geq 80.0\%$ Branch Coverage on core business services.
 - **GitHub Actions:** Automated pipeline on push/pull request.
 - **Jenkins:** Local containerized pipeline execution (`Jenkinsfile` + Docker Compose).
-

8. Final Deliverables Mapping

1. **GitHub Repository:** Complete codebase, unit/integration/system test suites, CI configurations, and `README.md`.
2. **Part A — Test Plan PDF:** Scope, approach, entry/exit criteria, risk-based prioritization, roles, schedule.
3. **Part B — Test Design Document PDF:** EP, BVA, Decision Tables, State Transitions with derived test cases.
4. **Parts F & G — Defect Log & Quality Metrics PDF:** Defect lifecycles, DRE (100%), defect density.
5. **Parts H & I — Test Summary Report & Reflection PDF:** Verification vs Validation, Error/Fault/Failure, release recommendation.